

GLOBAL PAYMENT MANAGEMENT SYSTEM

JOES AROCKIAM X

Register No: 2133020

**DISSERTATION SUBMITTED IN PARTIAL FULFILMENT
OF THEREQUIREMENTS FOR THE DEGREE OF
M.Sc. (DECISION AND COMPUTING
SCIENCES)OF ANNA UNIVERSITY**



November 2024

**DEPARTMENT OF COMPUTING
COIMBATORE INSTITUTE OF
TECHNOLOGY**

(Autonomous Institution affiliated to Anna University)

COIMBATORE – 641014

CONTENTS

CHAPTER	PAGE NO
ACKNOWLEDGEMENT	i
SYNOPSIS	ii
PREFACE	iii
I INTRODUCTION	4
1.1 ORGANIZATION PROFILE	4
1.2 PROBLEM DEFINITION	6
1.3 SYSTEM ENVIRONMENT	7
1.4 TOOLS AND TECHNOLOGIES USED	7
II SYSTEM ANALYSIS	9
2.1 PROPOSED SYSTEM	9
2.2 USE CASE MODEL	9
2.3 SOFTWARE REQUIREMENTS SPECIFICATION	13
2.4 TEST PLAN	16
III SYSTEM DESIGN	18
3.1 ARCHITECTURAL DESIGN	18
3.2 STRUCTURAL DESIGN	21
3.3 BEHAVIOURAL DESIGN	23
3.4 TABLE DESIGN	27
3.5 DEPLOYMENT DESIGN	40
3.6 USER INTERFACE DESIGN	41
IV SYSTEM TESTING	46
4.1 TEST CASES AND TEST REPORTS	46
V SYSTEM IMPLEMENTATION	51
VI CONCLUSION	52
BIBLIOGRAPHY	

ACKNOWLEDGEMENT

During the course of my Internship and in the process of preparing this project, I was helped and guided by many respected persons, who deserve my deepest gratitude.

I would first like to thank **Dr. A. RAJESWARI, Principal, Coimbatore Institute of Technology, Coimbatore** for her kind patronage.

I am highly indebted to **Dr. K SAKTHI MALA, Dean, Department of Computing, Coimbatore Institute of Technology** for her encouragement throughout this project

I would like to express my sincere gratitude and appreciation to **Dr. A. KANNAMMAL, Professor and Head of Department of Computing - Decision and Computing Sciences, Coimbatore Institute of Technology** for giving me an opportunity to take up this project.

I am also grateful to my guide **MS. K. VANI, Assistant Professor, Department of Computing - Decision and Computing Sciences, Coimbatore Institute of Technology** for supporting me throughout the execution of the project. I am thankful for her guidance, her invaluable constructive criticism and friendly advice during the project work.

I would thank **DUOEDGE Information systems, Chennai** for hiring me as an intern and giving me the opportunity for doing this project. As the completion of this internship gives me much pleasure, I am also extremely thankful to my industry guide, **Mr. Kathirvel Pandi J DEVOPS ENGINEER** for his encouragement throughout the internship period.

I would also like to thank my family, my colleagues at **DUOEDGE Information systems** and my peers at Coimbatore Institute of Technology for their constant support and encouragement.

SYNOPSIS

The Payment Management System is made to make the entire procedure of managing payment of a company's payment quite easy and efficient by providing a reliable payment processing, financial management and reporting services all in one seamless platform.

In the absence of a system integrated in the Payment Management platform, companies suffer from manual payment effort, problems of time lags in payment. Disparate approaches and absence or difficulty in making a proper auditing of payments made can also create room for inefficiencies and errors, not to mention net operational costs rising. When there is no integrated database, related information including customer database and transaction details, and relevant reports, poor decisions and poor financial visibility can occur.

These issues are overcome by the Payment Management System with its features that allow: processing of payments completed by the user, managing of transaction information of the above and generating reports on the financial transactions in detail as it happens. The system is integrated with various payment methods catering for both the businesses and their customers ease of use and flexibility. They are able to manage their payment history by tracking the communication relating to the payments, confirmations on payment and requests for reimbursement of the payments done from the system. In addition, the above system has worked to improve the reporting capabilities of the system where it is easy to report on various aspects of the transaction data, highlight some of its features, and help in maintaining financial discipline. They are also allowed to manage the overall activities of the system where they can manage users of the system, as well as oversee the various steps in payment processing.

PREFACE

CHAPTER I- INTRODUCTION

This chapter gives an introduction about the organization where the project was done. It also introduces the project, giving a description about the system that was built and the environment in which it was developed and deployed.

CHAPTER II - SYSTEM ANALYSIS

This chapter discusses the problem, with detailed analysis, system requirements and the specifications and it also covers the features of the proposed system. It also describes the functional and non-functional requirements of the system.

CHAPTER III - SYSTEM DESIGN

This chapter explains the design of the system that developed with related diagrams which outlines the different phases of the system development and elucidates the user interface flow.

CHAPTER IV - SYSTEM TESTING

This chapter explains the test cases, expected output and results.

CHAPTER V - SYSTEM IMPLEMENTATION

This chapter explains how the system is implemented, what are the pre-installed software's and walk-through on validations done.

CHAPTER VI - CONCLUSION

This chapter puts forth suggestions for future enhancements of the project

CHAPTER I

INTRODUCTION

1.1 ORGANIZATION PROFILE

Duoedge Information Systems is a leading provider of high-quality customer care solutions, established to deliver efficient and effective services. Located in chennai, Duoedge specializes in multi-channel customer care solutions, offering support through telephone, email, chat, and web self-service. Our primary focus is to enhance customer satisfaction and promote revenue retention for our clients.

From pre-sales support to comprehensive post-sales account coverage, Duoedge Information Systems tailors its services to meet the unique needs of each client, ensuring cost-effective solutions that drive business success. Operating in the IT Services and IT Consulting industry, we leverage advanced technologies and best practices to deliver exceptional customer care. Our commitment to understanding client requirements allows us to create customized strategies that not only resolve issues promptly but also foster long-term relationships with customers.

1.2 PROBLEM DEFINITION

In today's world of online transactions, many businesses and consumers face challenges due to the lack of a simple and secure way to manage payments. Current systems are often confusing and disconnected, leading to slow processing, difficulties in managing user accounts, and problems with reporting. This can result in unhappy customers and lost revenue for businesses.

Additionally, keeping up with rules and regulations related to payment security and transparency has become more complicated. Many organizations find it hard to comply with these requirements, which can lead to financial risks and damage to their reputation. Without a complete solution that makes payment processes easier and safer, it becomes difficult for businesses to provide a great customer experience.

Therefore, there is a clear need for an effective Global Payment Management System. This system should simplify how payments are handled and improve user experience by enhancing security, enabling quick processing, and providing useful insights. By solving these issues, the proposed system aims to improve payment processing standards in the industry and help businesses succeed in the competitive online market.

1.2.1 OBJECTIVE

Automated Deployment and CI/CD Pipelines

Objective: To make the development and release process faster and smoother, automated deployment and CI/CD pipelines are used. This ensures that updates and new features are delivered quickly and reliably, keeping the system efficient and minimizing downtime.

Containerization and Maintaining

Objective: By using Docker for containerization and Jenkins, the system can run consistently across different platforms. This setup helps manage resources better, allows easy scaling, and speeds up recovery, making the payment system more reliable and secure.

Robust Database Design

Objective: A well-designed database provides a strong and secure structure for storing payment data. It's organized for quick data access, easy scaling, and secure transactions, helping maintain the system's reliability and support a great user experience.

1.2.2 SCOPE

The project scope involves implementing essential DevOps practices like automated deployment, CI/CD pipelines, and container orchestration to build a reliable and scalable payment management system. Using Docker and Kubernetes ensures consistent operation and efficient resource management, while AWS deployment provides robust infrastructure support. This project aims to streamline deployment, enhance security, and ensure continuous system availability. A well-designed database enables secure, real-time transaction handling and reporting, creating a dependable platform for delivering secure, fast, and efficient payment services at scale.

1.2.3 USERS

The primary users of the Global Payment Management System are Business Customers, End Users, and Administrators. Business Customers are companies and merchants who utilize the platform to manage and process payments efficiently. End Users are individual consumers who engage in transactions through the system. Administrators are responsible for overseeing the platform's management, ensuring security, monitoring transactions, and maintaining system performance.

1.2.4 BENEFITS

- **Streamlined Payment Processing:** The system allows businesses to manage their payment transactions in real-time, thus eliminating delay and ensuring that transactions are completed much faster than traditional systems.
- **Enhanced Security:** With sophisticated security protocols, the platform ensures that payment data is well taken care of, hence securing users and businesses against fraud and unauthorized access, which gives confidence in the system.
- **It ensures scalability and flexibility,** by using containerization and then AWS deployment so that whenever high transaction volume or more business comes on board with such systems it can readily adapt without trading off their efficiency.
- **Centralized Transaction Management:** In a central place, business will have the power to handle all its transactions for payments hence streamlines work and minimizes intricacies in managing various means of payments.
- **Administrators control :** Administrators could be able to govern the transaction, monitor overall performance, and be in charge of the upgrade processes ensuring the platform is secure and efficient for all users who depend on it.

1.3 SYSTEM ENVIRONMENT

This section describes the system environment used for the development of the Global Payment Management System project.

1.3.1 DEVELOPMENT ENVIRONMENT

SOFTWARE ENVIRONMENT

Operating System: Linux Ubuntu

Technology: Java

Database: MySQL 5.7

Web Browser: For testing and user interface display

HARDWARE ENVIRONMENT

Processor: AMD Ryzen

RAM: 8GB RAM

HDD: SSD 256

1.3.2 TOOLS AND TECHNOLOGIES USED

This section describes the tools and technologies used in the project along with a brief description of each.

Frontend

1. HTML/CSS: Standard web technologies for structuring content and defining styles for the user interface.
2. JavaScript: A programming language used to create dynamic content on web pages.
3. jQuery: A fast, small, and feature-rich JavaScript library that simplifies HTML document traversing, event handling, and AJAX interactions.
4. Bootstrap: A front-end framework for developing responsive and mobile-first websites.

Backend

1. Java Servlets: Java classes that handle HTTP requests and responses, serving as the

backbone of the web application.

2. JSP (JavaServer Pages): A technology for developing web pages that contain dynamic content, allowing for a simplified approach to building user interfaces.
3. DAO Layer: Data Access Object pattern used to interact with the database, executing SQL queries and managing data retrieval and storage.
4. MySQL 5.7: A relational database management system used for storing and managing payment transaction data securely.

Containerization and Orchestration

1. Docker: A platform that enables developers to automate the deployment of applications inside lightweight containers, ensuring consistent environments.
2. Kubernetes: An open-source system for automating the deployment, scaling, and management of containerized applications.

Continuous Integration/Continuous Deployment (CI/CD)

Jenkins: An open-source automation server used to automate the building, testing, and deployment of applications, facilitating CI/CD practices.

Deployment and Hosting

AWS (Amazon Web Services): A cloud platform that provides various services for hosting the application, offering scalability, reliability, and security for production environments.

CHAPTER II

SYSTEM ANALYSIS

System analysis is a process of studying the system in order to identify the goals, purpose of the system and a better understanding of the system's requirements. This chapter gives a brief discussion about the detailed study of the proposed system and the different functionalities involved in the application.

2.1 PROPOSED SYSTEM

Introducing the innovative Global Payment Management System, a robust platform meticulously designed to streamline payment processing and management. This user-friendly system allows users to securely execute and track transactions, manage payment methods, and access transaction histories with ease. Its intuitive interface ensures a hassle-free experience, enabling users to navigate through functionalities such as payment processing, reporting, and user management seamlessly. Whether a business owner or an administrator, this platform is tailored to meet specific needs, eliminating the complexities of payment management. With the Global Payment Management System, users can focus on their core activities, confident that the system will handle all payment-related tasks efficiently.

2.2 USE CASE MODEL

A use case diagram is a visual representation of the interactions between users and the system, illustrating the relationship between different user roles and the various functionalities they can access. The following sections outline the primary use cases for both regular users and administrators within the Global Payment Management System.

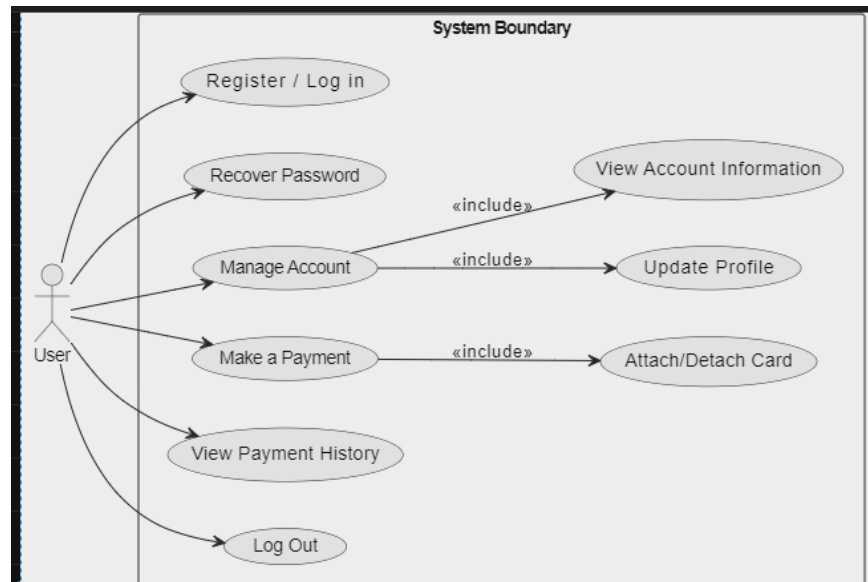


Figure 2.1 Use case diagram for users

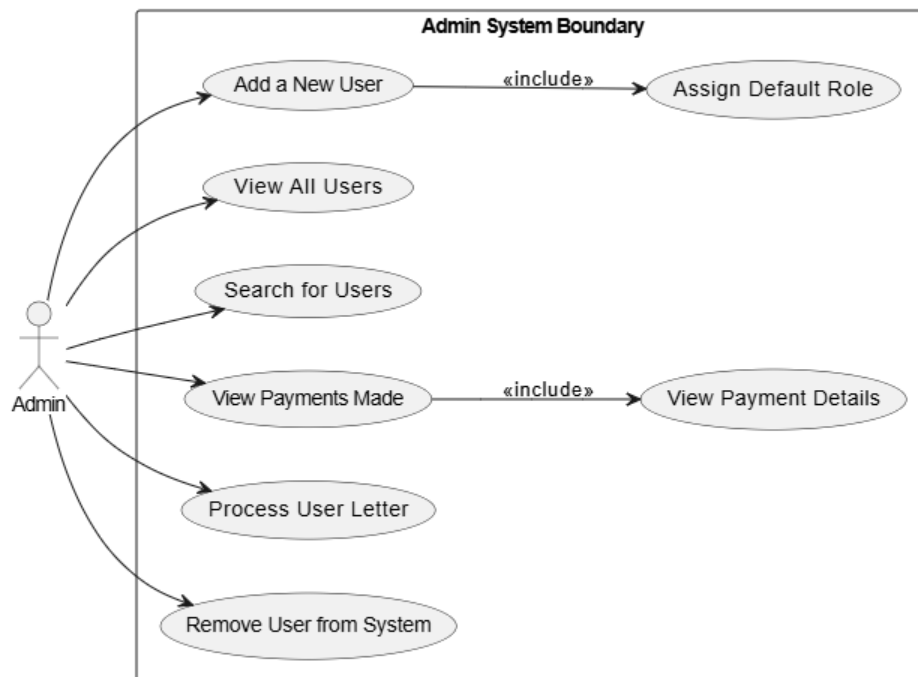


Figure 2.2 Use case diagram for Admin

2.2.1 EXTENDED USE CASE

A. USER USE CASES

Goal	Input	Precondition	Postcondition	Exception
Authenticate User	Email, Password	User is registered	User is logged into the system	Insufficient credentials
Log in to Account	Login credentials	User is registered	User is logged into their account	Invalid login data
Recover Password	Email, recovery info	User forgot password	Password recovery link sent to user's email	Invalid recovery information
Update Password	Old password, New password	User is logged in	Password updated successfully	Incorrect old password
Clear Action Log	Confirmation to clear log	User is logged in	Action log is cleared	Failure to clear log
Search for Actions	Search criteria	User is logged in	Relevant actions are displayed	No actions found
Detach Card	Card details	User is logged in	Card is detached	Cannot detach card
Block Card	Card details	User is logged in	Card is blocked	Cannot block card
Unblock Card	Card details	User is logged in	Card is unblocked	Cannot unblock card
Update Personal Data	New personal data	User is logged in	User data updated	Invalid data
View Attached Cards	-	User is logged in	List of attached cards displayed	No cards attached
View Action Log Info	-	User is logged in	Action logs displayed	No logs available
Make Payment	Payment details	User is logged in	Payment processed successfully	Invalid payment details
Repeat Payment	Payment details	User is logged in	Payment repeated successfully	Invalid payment details

B. ADMIN USE CASES

Goal	Input	Precondition	Postcondition	Exception
Add a New User	User details	Admin is logged in	New user created in the system	Invalid user details
View All Users	-	Admin is logged in	List of all users displayed	No users registered
Search for Users	Search criteria	Admin is logged in	Relevant users displayed	No users match criteria
View Payments Made	-	Admin is logged in	List of all payments displayed	No payments made
Block User Account	User details	Admin is logged in	User account blocked	Cannot block account
Unblock User Account	User details	Admin is logged in	User account unblocked	Cannot unblock account
Process User Letter	User letter	Admin is logged in	Inquiry processed and responded	Cannot process letter
Remove User from System	User details	Admin is logged in	User account removed	Cannot remove user

2.3 SOFTWARE REQUIREMENTS SPECIFICATION

2.3.1 FUNCTIONAL REQUIREMENTS

A functional requirement document defines the necessary functions of a system or its subsystems based on the type of software, user expectations, and system environment. Functional requirements for the Payment Management System include:

- User Registration and Profiles
- Account and Transaction Management
- Payment Processing
- Transaction History and Statements
- Fraud Detection and Alerts
- Admin Dashboard and User Management
- Search and Filter Functionality
- Notifications and Alerts
- Responsive Design

User Registration and Profiles

Allow users to register and create profiles with relevant information. Enable users to manage account settings and security preferences.

Account and Transaction Management

Provide functionalities for users to manage their accounts, link multiple payment methods, and view account details. Enable secure transaction handling for deposits, transfers, and withdrawals.

Payment Processing

Integrate a secure payment gateway for processing transactions. Ensure seamless handling of transactions, including credit/debit processing, bank transfers, and online payments.

Transaction History and Statements

Allow users to view detailed transaction history and generate statements. Provide filters based on date, transaction type, or amount for easier access.

Fraud Detection and Alerts

Implement algorithms for fraud detection. Alert users of any suspicious activities and provide options for reporting suspicious transactions.

Admin Dashboard and User Management

Enable admins to monitor user activities, review suspicious transactions, and manage user accounts. Include access to view system logs and activity summaries.

Search and Filter Functionality

Implement advanced search and filter options for easy access to user accounts, transactions, and reports based on various.

Notifications and Alerts

Send notifications to users regarding successful transactions, failed transactions, account changes, or fraud alerts.

Responsive Design

Ensure the application is designed with a responsive layout, delivering a seamless user experience on desktops, tablets, and mobile devices.

2.3.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the general characteristics of a system. For the Payment Management System, the non-functional requirements include

- Security
- Reliability
- Usability
- Performance
- Scalability and Availability
- Continuous Integration and Deployment

Security

Implement strong encryption for sensitive data, including user credentials and transaction details.

Require multi-factor authentication (MFA) for high-security transactions.

Enforce strict password policies to prevent unauthorized access.

Ensure secure storage and transmission of sensitive information to prevent data breaches.

Reliability

Maintain a low failure rate by implementing error detection and handling mechanisms.

Regularly perform system audits and monitor uptime to ensure a highly reliable platform.

Utilize robust backup and recovery processes to protect data integrity.

Usability

Design an intuitive and user-friendly interface (UI) with accessible menus and clear action buttons.

Provide accessible help sections and customer support options.

Performance

Ensure the system can handle a high volume of transactions without significant lag.

Optimize loading times for pages and transaction history, especially during peak usage times.

Regularly monitor system performance and optimize databases and queries for faster transaction processing.

Scalability and Availability

Utilize load balancing to manage heavy user traffic and ensure availability.

Implement auto-scaling with cloud services to accommodate user growth.

Ensure high availability by deploying on a reliable cloud platform with a multi-region failover setup.

Continuous Integration and Deployment (CI/CD)

Jenkins: Integrate Jenkins for automated build and testing processes to ensure continuous integration and quick detection of bugs.

Docker: Use Docker to create containerized environments for consistent and isolated application deployments.

Kubernetes: Deploy the application using Kubernetes for efficient management of Docker containers, auto-scaling, and high availability across distributed environments.

2.4 TEST PLAN

The Test Plan outlines the strategy to verify that the Payment Management System meets all specified requirements and functions correctly. This plan is prepared by software testers to ensure quality and reliability.

2.4.1 TEST SCOPE

All functionalities of the Payment Management System are within the test scope to ensure the system operates as intended. Testing will focus on identifying bugs, errors, or inconsistencies in the application, enabling developers to address any issues. This process will provide insights into potential areas for improvement and offer best practice recommendations to end users to minimize errors in application use.

2.4.2 TESTING TECHNIQUES

MANUAL TESTING

Manual Testing finds out the defects or bugs in the application. Testing manually plays an important role for the end-user and verifies that all the features of the application are working correctly. Test cases are executed manually without using any automation tools. And problems faced by customers are rectified manually.

UNIT TESTING

Unit Testing is done at the end of each development level to test particular functions and sub-modules. It is a software development process in which the smallest testable parts of an application, called units, are individually scrutinized for proper operation.

INTEGRATION TESTING

Integration testing is done to validate that different software components, subsystems, or applications work together as a system to achieve the desired functionality and performance. It involves checking individual components or units of a software project to expose defects and problems to verify that they work together as designed.

Automated Testing with SonarQube and Jenkins

Automated testing plays a crucial role in efficiently identifying bugs, code smells, and security vulnerabilities in the system. Using SonarQube integrated with Jenkins builds, we conduct continuous code analysis and quality checks during each build. This integration allows for automated, consistent quality checks on the codebase, ensuring any issues are detected promptly.

SonarQube: Analyzes code quality, detects code smells, and highlights potential security risks. The tool provides a detailed report on areas needing improvement, helping to enforce coding standards.

Jenkins: Triggers automated SonarQube tests during the build process, ensuring any new code meets quality standards before deployment. Jenkins builds enable continuous integration and automated testing, leading to more reliable and robust releases.

CHAPTER III

SYSTEM DESIGN

This chapter explains the process of transforming requirements into a structured representation of software. It defines the components of the software, their relationships, and provides a blueprint for implementation, focusing on achieving both functional and non-functional requirements.

3.1 ARCHITECTURAL DESIGN

Architectural design defines the structure of data, hardware, and software components, and their interfaces, to establish a framework for the system. This architecture supports the requirements of the system while focusing on scalability, modularity, and efficiency. Below is an overview of the components based on the architectural design of the Payment Management System

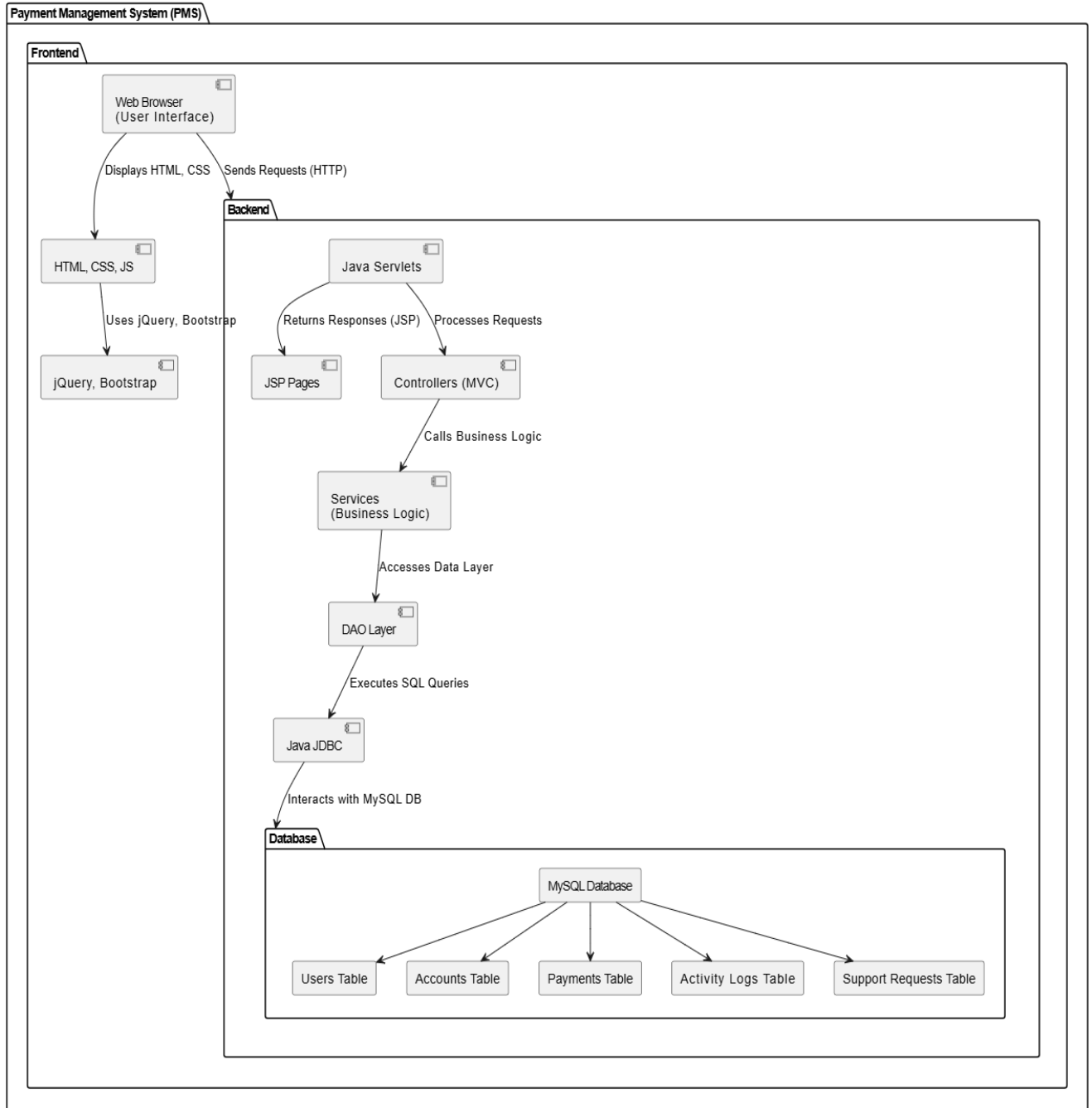


Figure 3.1 Architectural design

FRONTEND

1. Web Browser (User Interface)
 - Acts as the client interface, allowing users to interact with the system.
 - Sends HTTP requests to the backend and displays HTML and CSS content.
2. Technologies Used
 - HTML, CSS, JavaScript: Provides structure, styling, and interactivity.
 - jQuery, Bootstrap: Enhances UI components, providing responsive and interactive design elements.

BACKEND

Java Servlets

- Processes HTTP requests from the frontend and returns appropriate responses.
- Uses Java Servlets to interact with the system's controllers and respond with dynamic JSP pages.

JSP Pages

- Generates dynamic web content and displays data to the user.
- Responsible for rendering user-facing components based on data received from the backend.

Controllers (MVC)

- Manages the flow of information between the user interface and backend services.
- Routes requests to the appropriate services in the system for processing.

Services (Business Logic)

- Implements core business logic and operations of the Payment Management System.
- Contains rules and processing logic for handling transactions, user requests, and application services.

DAO Layer

- Provides a Data Access Object (DAO) interface to interact with the database.
- Facilitates communication between the services and the database by executing SQL queries.

Java JDBC

- Utilizes Java Database Connectivity (JDBC) to interact with the MySQL database. Ensures data consistency and performs CRUD operations on the database.

DATABASE

MySQL Database

Stores and manages persistent data for the application, including user accounts, payment details, activity logs, and support requests

Database Tables

- Users Table: Stores user information, such as login credentials and personal details.
- Accounts Table: Contains account information related to users and payment accounts.
- Payments Table: Manages records of transactions and payments made through the system.
- Activity Logs Table: Logs user actions and system activities for tracking and audit purposes.
- Support Requests Table: Stores user support inquiries and requests for administrative review.

3.2 STRUCTURAL DESIGN

Structured design breaks a system down into functional modules. Each module can be described as having inputs, processing, and outputs. Structured design is a top-down decomposition of system functionality, while object oriented design focuses on system behavior.

3.2.1 MODULE DIAGRAM

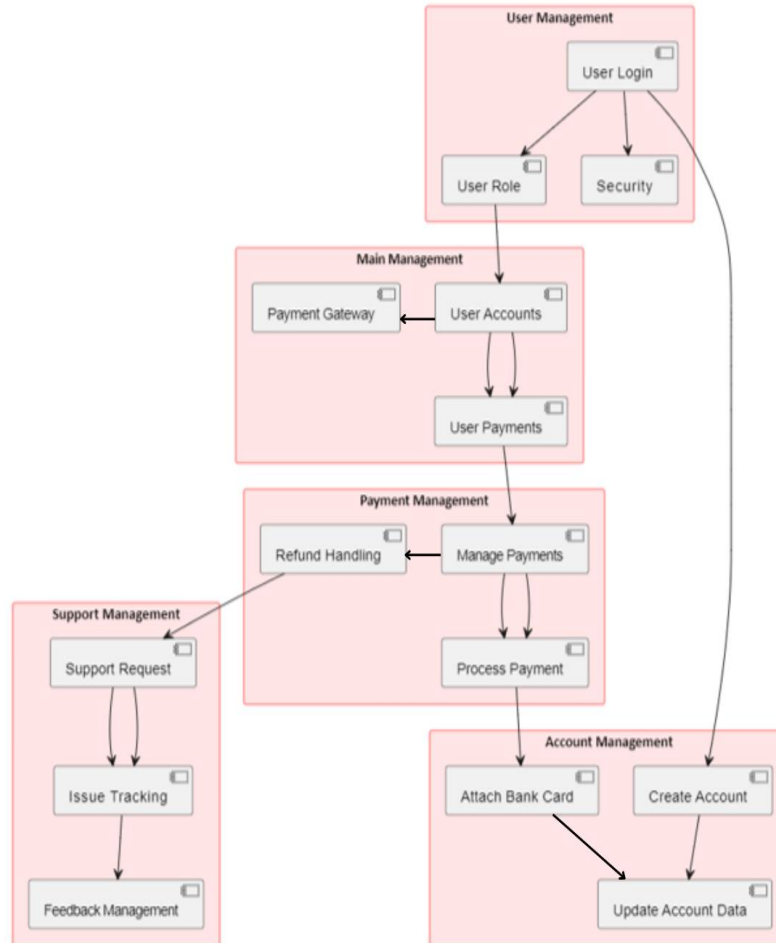


Figure 3.2 Module Design of GPMS

- **User Management:** Manages user authentication and roles, with components for user login, security, and role-based access control.
- **Main Management:** Acts as a central hub, connecting user accounts, payment gateways, and user payments.

- **Payment Management:** Handles all payment processes, including managing and processing payments as well as refund handling.
- **Support Management:** Addresses customer support with components for support requests, issue tracking, and feedback management.
- **Account Management:** Manages account functions such as creating accounts, attaching bank cards, and updating account information.

3.3 BEHAVIOURAL DESIGN

The activity diagram represents the behavior of the system. It shows the workflow and step-by-step activities and actions performed. It also describes the business processes, use cases and to document the implementation of the system process.

3.3.1 ACTIVITY DIAGRAM

The activity diagram represents the behavior of the system. It shows the workflow and step-by-step activities and actions performed. It also describes the business processes, use cases and to document the implementation of the system process.

Activity Diagrams describe how activities are coordinated to provide a service which can be at different levels of abstraction. Typically, an event needs to be achieved by some operations, particularly where the operation is intended to achieve a number of different things that require coordination, or how the events in a single use case relate to one another, in particular, use cases where activities may overlap and require coordination.

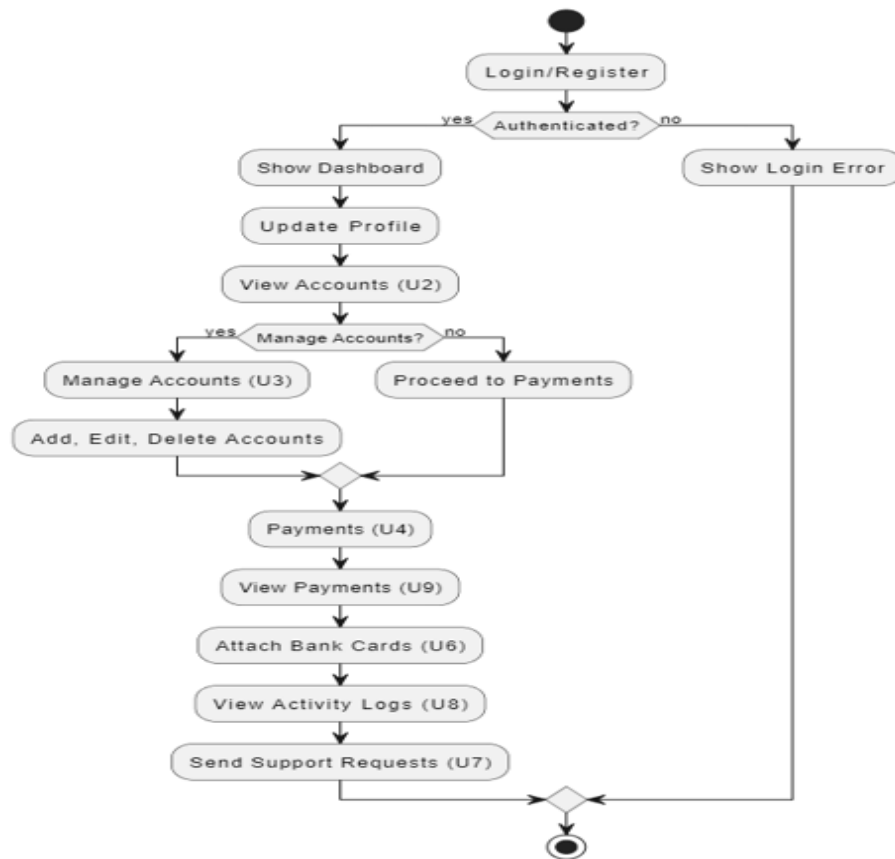


Figure 3.3 Activity Diagram for User

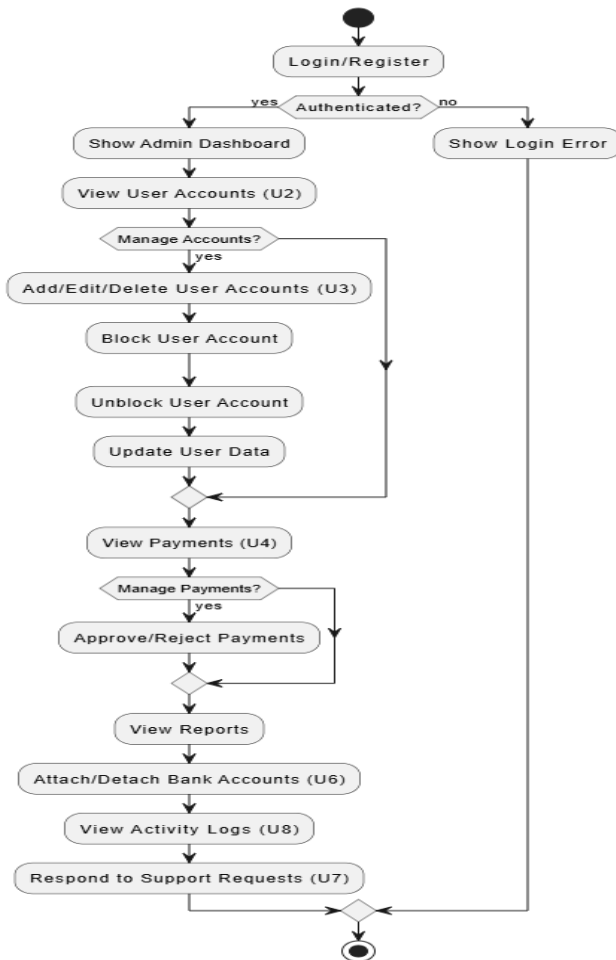


Figure 3.4 Activity Diagram for Admin

3.3.2 SEQUENCE DIAGRAM

Sequence Diagrams captures the interaction that takes place in a collaboration that either realizes a use case or an operation. It shows the high-level interactions between user of the system and the system, between the system and other systems, or between subsystems.

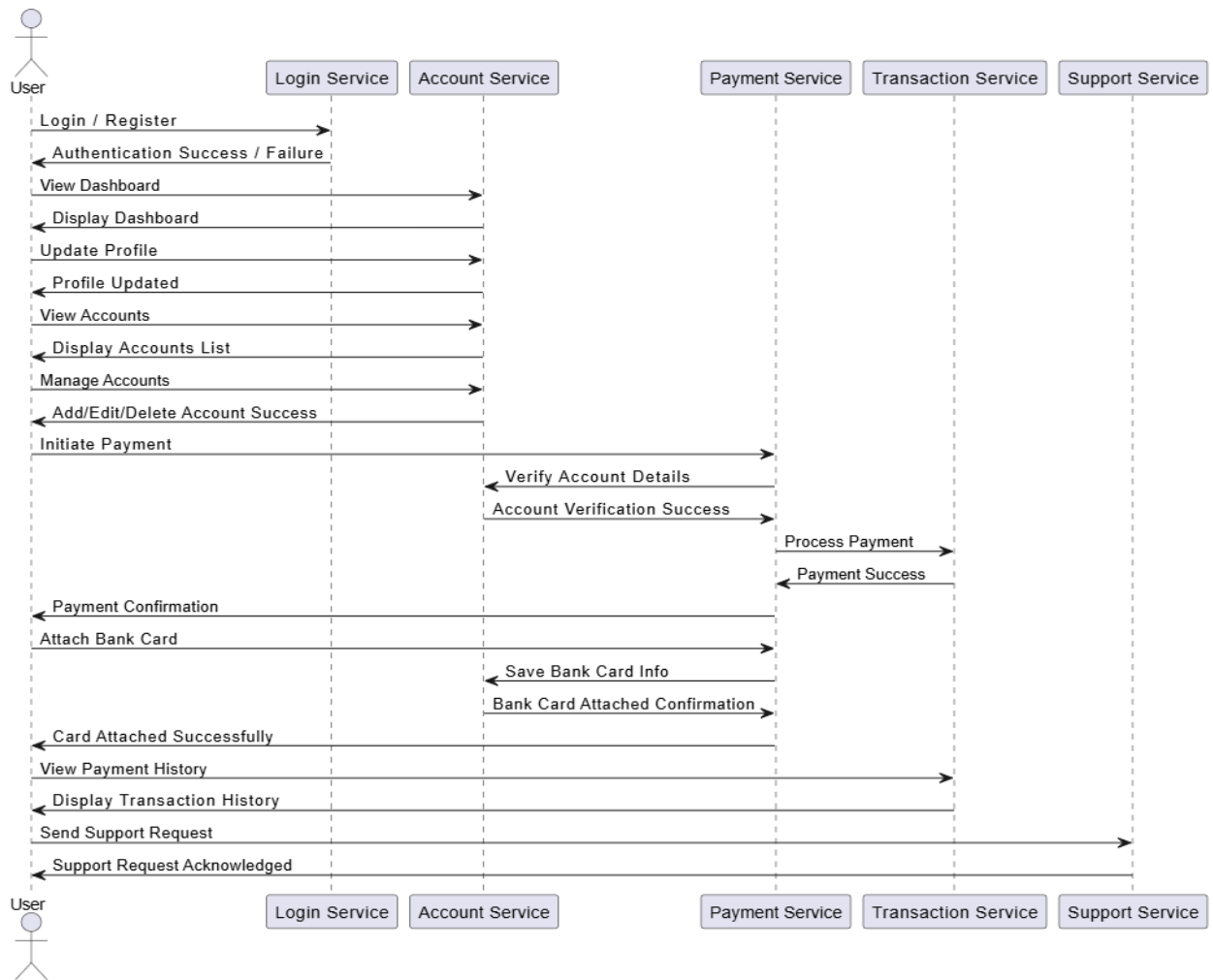


Figure 3.5 Sequence Diagram for user

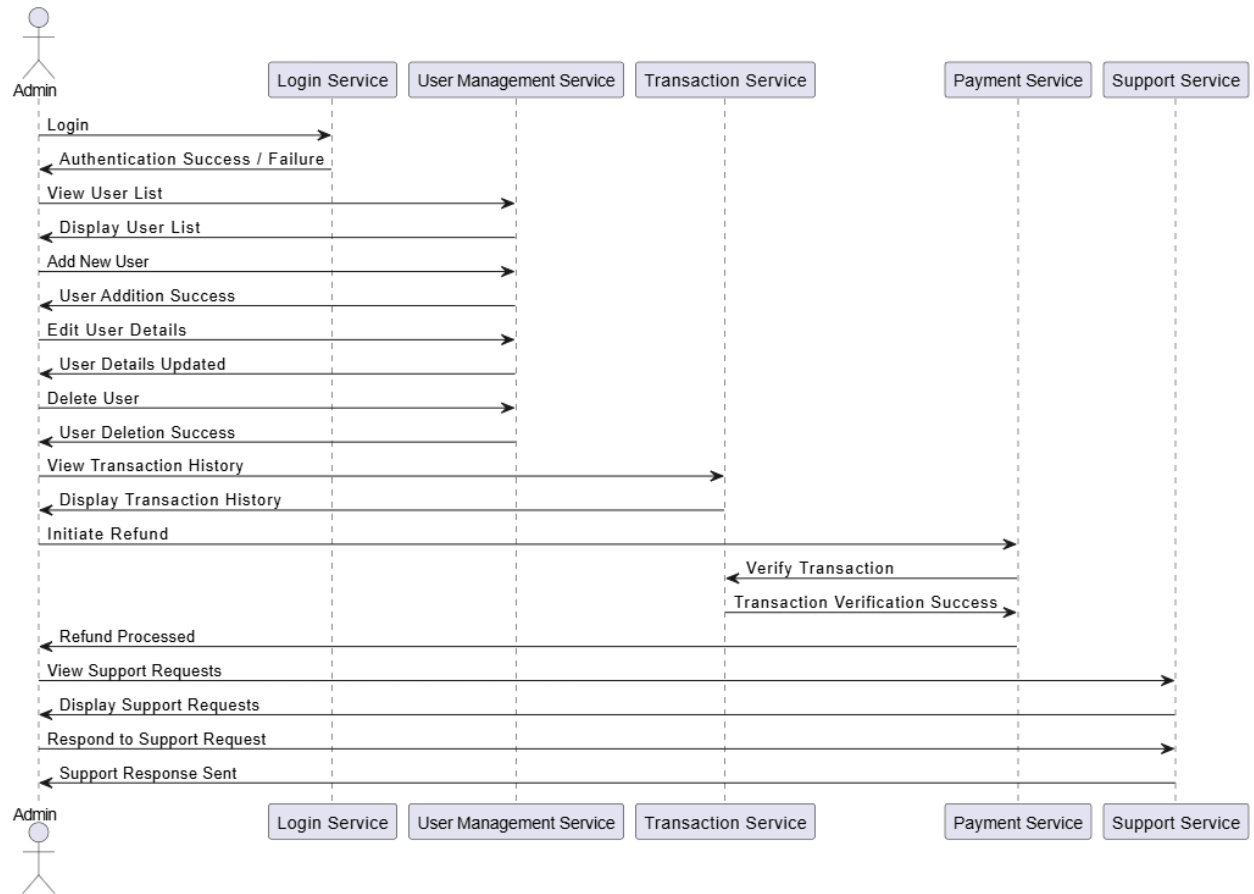


Figure 3.6 Sequence Diagram for Admin

3.4 TABLE DESIGN

Table-driven design is an approach to software development engineering that is aimed at simplifying and generalizing applications by separating the program control variables and parameters from the code and placing those in separate external tables. The main objective is to decouple the program control data from the application logic and to put emphasis on modularity to ease change management.

3.4.1 ENTITY RELATIONSHIP DIAGRAM

Entity-Relationship Diagram, it is used to analyze the structure of the Database. It shows relationships between entities and their attributes. An ER Model provides a means of communication.

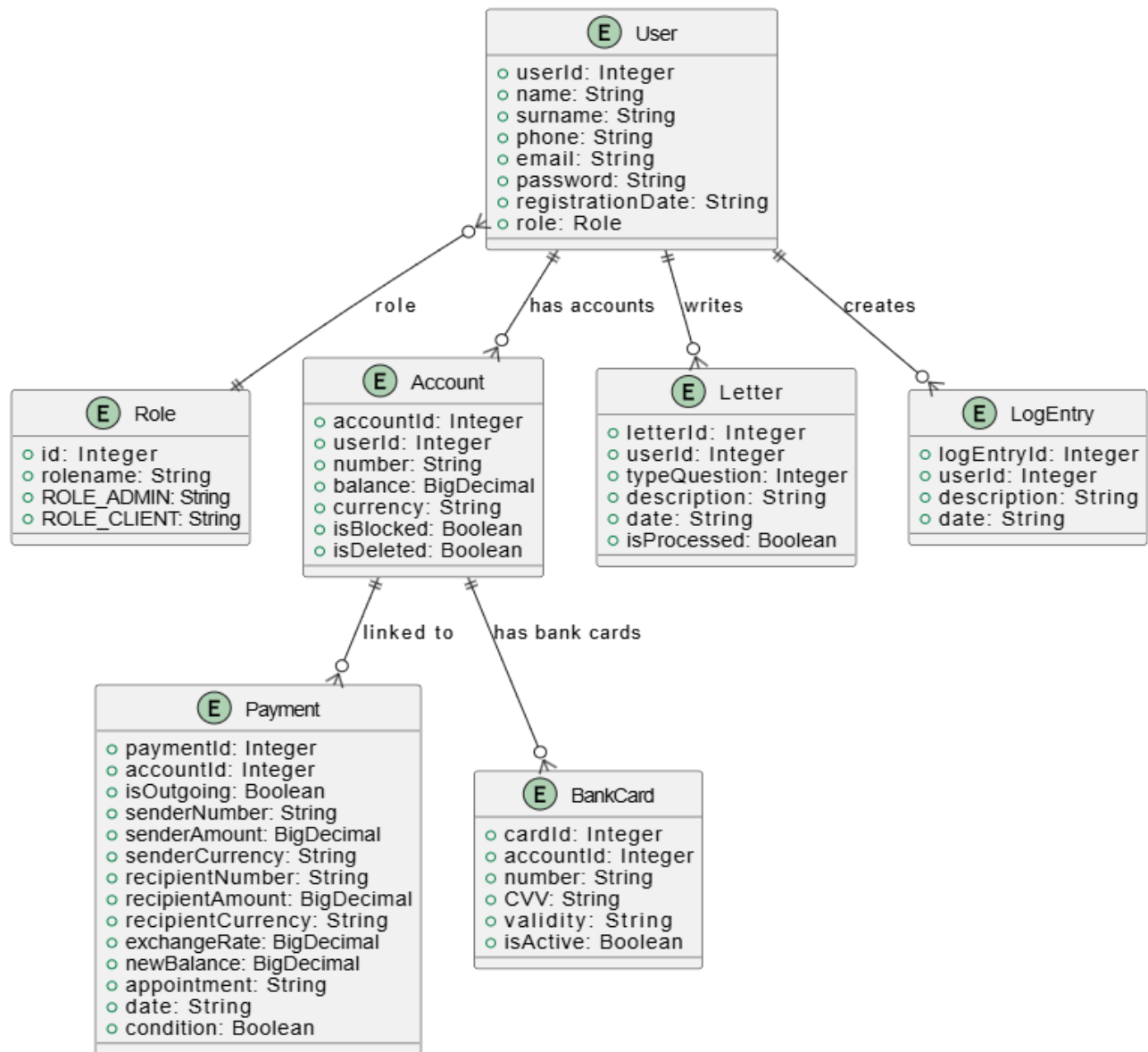


Figure 3.7 Entity Relationship Diagram of GPMS

Table 3.4.1 - Roles Table

Field	Description	Data Type	Required?
id	Unique identifier for the role	INT	Yes (Primary Key)
name	Name of the role (e.g., administrator, user)	VARCHAR	Yes

Table 3.4.2 - Users Table

Field	Description	Data Type	Required?
id	Unique user ID	INT	Yes (Primary Key)
name	User's real name	VARCHAR	Yes
surname	User's surname	VARCHAR	Yes
phone_number	User's phone number (formatted by country)	VARCHAR	Yes
email	User's email address (used for communication and recovery)	VARCHAR	No
password	Encrypted user password using MD5 hashing	VARCHAR	Yes
registration_date	Date of user registration (auto-set by system)	DATE	Yes
role_id	Role ID (links to Roles table)	INT	Yes (Foreign Key)

Table 3.4.3 - Action Log Table

Field	Description	Data Type	Required?
log_entry_id	Unique identifier for log entry	INT	Yes (Primary Key)
user_id	ID of the user performing the action	INT	Yes (Foreign Key)
description	Description of the action	TEXT	Yes
date	Date and time when the action was performed	TIMESTAMP	Yes

Table 3.4.4 - Letters Table

Field	Description	Data Type	Required?
letter_id	Unique identifier for the email	INT	Yes (Primary Key)
user_id	ID of the user sending the letter	INT	Yes (Foreign Key)
question_type	Type of question selected by user	VARCHAR	Yes
description	Description of the situation, problem, or suggestion	TEXT	No
date	Date when the letter was sent	DATE	Yes
email_processing_rate	Indicator if the email has been processed (1 for processed)	BOOLEAN	Yes

Table 3.4.5 - Accounts Table

Field	Description	Data Type	Required?
account_id	Unique identifier for the account	INT	Yes (Primary Key)
user_id	ID of the user associated with the account	INT	Yes (Foreign Key)
number	Unique 20-digit account number	VARCHAR	Yes
balance	Current balance in the account	FLOAT	Yes
currency	ISO 4217 currency code	VARCHAR	Yes
block_status_indicator	Boolean indicating if the account is blocked (1 = blocked)	BOOLEAN	Yes
deletion_status_indicator	Boolean indicating account visibility (1 = deleted)	BOOLEAN	Yes

Table 3.4.6 - Bank Cards Table

Field	Description	Data Type	Required?
card_id	Unique identifier for the bank card	INT	Yes (Primary Key)
account_id	ID of the associated account	INT	Yes (Foreign Key)
number	16-digit card number	VARCHAR	Yes
cvv_code	CVV code for card verification	VARCHAR	Yes
validity	Expiration date of the card	DATE	Yes
activity_status_score	Boolean indicating if the card is active (1 = active)	BOOLEAN	Yes

3.4.3 Development of Database Queries

In the developed database structure, a range of queries with different complexities were written. Most of these queries perform the four essential data management functions: Create, Read, Update, and Delete (CRUD).

This system uses several design patterns common in large-scale projects, including the Data Access Object (DAO) pattern. The DAO pattern helps separate the application layer from the database layer by providing an abstract API. This API simplifies CRUD operations and shields the program from the complexities of the database. Below are details on the queries written in each DAO class, with explanations and examples provided in the form of screenshots.

User Dao Implement Class

The UserDaoImpl class implements the methods in the User Dao interface, handling user data operations. The class includes the following queries:

CREATE_USER: Creates a new user account.

UPDATE_USER: Updates user details.

DELETE_USER: Completely removes a user, along with associated data (like logs and accounts).

FIND_USER_BY_ID: Searches for a user by their ID.

FIND_USER_BY_LOGIN_PASSWORD: Finds a user by login and password.

FIND_USER_BY_PHONE: Finds a user by phone number.

FIND_ALL_USERS: Retrieves all users in the system.

SEARCH_BY_CRITERIA: Searches for users based on specific criteria.

```
public class UserDaoImpl implements UserDao {  
    /**  
     * SQL queries  
     */  
    private static final String CREATE_USER =  
        "INSERT INTO users (name, surname, phone, email, password, registration_date, role_id) " +  
        "VALUES(?, ?, ?, ?, ?, ?, ?)";  
    private static final String UPDATE_USER =  
        "UPDATE users SET name = ?, surname = ?, phone = ?, email = ?, password = ? " +  
        "WHERE user_id = ?";  
    private static final String DELETE_USER = "DELETE FROM users WHERE user_id = ?";  
    private static final String FIND_USER_BY_ID =  
        "SELECT users.*, roles.title FROM users " +  
        "INNER JOIN roles ON users.role_id = roles.id " +  
        "WHERE users.user_id = ?";  
    private static final String FIND_USER_BY_LOGIN_PASSWORD =  
        "SELECT users.*, roles.title FROM users " +  
        "INNER JOIN roles ON users.role_id = roles.id " +  
        "WHERE users.phone = ? AND users.password = ?";  
    private static final String FIND_USER_BY_PHONE =  
        "SELECT users.*, roles.title FROM users " +  
        "INNER JOIN roles ON users.role_id = roles.id " +  
        "WHERE users.phone = ?";  
    private static final String FIND_ALL_USERS =  
        "SELECT users.*, roles.title FROM users " +  
        "INNER JOIN roles ON users.role_id = roles.id";  
    private static final String SEARCH_BY_CRITERIA =  
        "SELECT users.*, roles.title FROM users " +  
        "INNER JOIN roles ON users.role_id = roles.id " +  
        "WHERE role_id = 1 AND name LIKE CONCAT(?, '%') AND surname LIKE CONCAT(?, '%') AND " +  
        "phone LIKE CONCAT(?, '%') AND email LIKE CONCAT(?, '%') ORDER BY registration_date DESC";  
}
```

Figure 3.8 – Request code for retrieving and manipulating user data

Action LogDao Implement Class

The ActionLogDao Implement class implements methods in the ActionLogDao interface to handle activity log data. The queries include:

CREATE_LOG_ENTRY: Adds a new entry to the log.

CLEAR_ACTION_LOG: Clears a specific user's log.

FIND_LOG_ENTRY_BY_ID: Finds a log entry by its ID.

FIND_LOG_ENTRIES_BY_USER_ID: Retrieves all log entries for a user.

SEARCH_BY_CRITERIA: Searches logs by specific criteria.

SEARCH_BY_CRITERIA_AND_FINAL_DATE_AS_CURRENT_TIMESTAMP: Searches logs by criteria with a condition to use the current timestamp for the end date.

```
public class ActionLogDaoImpl implements ActionLogDao {  
    /**  
     * SQL queries  
     */  
    private static final String CREATE_LOG_ENTRY =  
        "INSERT INTO action_log(user_id, description, date) " +  
        "VALUES(?, ?, ?)";  
    private static final String CLEAR_ACTION_LOG = "DELETE FROM action_log WHERE user_id = ?";  
    private static final String FIND_LOG_ENTRY_BY_ID = "SELECT * FROM action_log WHERE log_entry_id = ? ORDER BY date DESC";  
    private static final String FIND_LOG_ENTRIES_BY_USER_ID = "SELECT * FROM action_log WHERE user_id = ? ORDER BY date DESC";  
    private static final String SEARCH_BY_CRITERIA =  
        "SELECT * FROM action_log WHERE user_id = ? AND date BETWEEN " +  
        "STR_TO_DATE(?, '%d/%m/%Y %H:%i:%s') AND " +  
        "STR_TO_DATE(?, '%d/%m/%Y %H:%i:%s') ORDER BY date DESC";  
    private static final String SEARCH_BY_CRITERIA_AND_FINAL_DATE_AS_CURRENT_TIMESTAMP =  
        "SELECT * FROM action_log " +  
        "WHERE user_id = ? AND date BETWEEN " +  
        "STR_TO_DATE(?, '%d/%m/%Y %H:%i:%s') AND " +  
        "CURRENT_TIMESTAMP() ORDER BY date DESC";  
}
```

Figure 3.9 Request code for retrieving and manipulating action log data

LetterDao Implement Class

The LetterDao Implement class implements methods in the LetterDao interface to manage sent emails. The queries include:

CREATE_LETTER: Adds a new email.

UPDATE_LETTER: Updates email details (e.g., processing status).

DELETE_LETTER: Removes an email from the system.

FIND_LETTER_BY_LETTER_ID: Finds an email by its ID.

FIND_LETTERS_BY_USER_ID: Retrieves all emails for a specific user.

FIND_ALL_LETTERS: Retrieves all emails in the system.

SEARCH_BY_CRITERIA: Searches for emails based on criteria.

SEARCH_BY_CRITERIA_AND_FINAL_DATE_AS_CURRENT_TIMESTAMP: Searches emails by criteria with the end date as the current timestamp.

SEARCH_BY_CRITERIA_WITHOUT_TYPE_QUESTION: Searches emails by criteria, ignoring question type.

SEARCH_BY_CRITERIA_WITHOUT_TYPE_QUESTION_AND_FINAL_DATE_AS_CURRENT_TIMESTAMP: Searches emails by criteria, ignoring question type and setting the end date as the current timestamp.

```

public class LetterDaoImpl implements LetterDao {

    /**
     * SQL queries
     */
    private static final String CREATE_LETTER =
        "INSERT INTO letters(user_id, typeQuestion, description, date, is_processed) " +
        "VALUES(?, ?, ?, ?, ?)";
    private static final String UPDATE_LETTER = "UPDATE letters SET is_processed = ? WHERE letter_id = ?";
    private static final String DELETE_LETTER = "DELETE FROM letters WHERE letter_id = ?";
    private static final String FIND_LETTER_BY_LETTER_ID = "SELECT * FROM letters WHERE letter_id = ?";
    private static final String FIND_LETTERS_BY_USER_ID = "SELECT * FROM letters WHERE user_id = ?";
    private static final String FIND_ALL_LETTERS = "SELECT * FROM letters ORDER BY date ASC";
    private static final String SEARCH_BY_CRITERIA =
        "SELECT * FROM letters " +
        "WHERE is_processed = 0 AND typeQuestion LIKE ? AND date BETWEEN " +
        "STR_TO_DATE(?, '%d/%m/%Y %H:%i:%s') AND " +
        "STR_TO_DATE(?, '%d/%m/%Y %H:%i:%s') ORDER BY date ASC;";
    private static final String SEARCH_BY_CRITERIA_AND_FINAL_DATE_AS_CURRENT_TIMESTAMP =
        "SELECT * FROM letters " +
        "WHERE is_processed = 0 AND typeQuestion LIKE ? AND date BETWEEN " +
        "STR_TO_DATE(?, '%d/%m/%Y') AND " +
        "CURRENT_TIMESTAMP() ORDER BY date ASC;";
    private static final String SEARCH_BY_CRITERIA_WITHOUT_TYPE_QUESTION =
        "SELECT * FROM letters " +
        "WHERE is_processed = 0 AND date BETWEEN " +
        "STR_TO_DATE(?, '%d/%m/%Y %H:%i:%s') AND " +
        "STR_TO_DATE(?, '%d/%m/%Y %H:%i:%s') ORDER BY date ASC;";
    private static final String SEARCH_BY_CRITERIA_WITHOUT_TYPE_QUESTION_AND_FINAL_DATE_AS_CURRENT_TIMESTAMP =
        "SELECT * FROM letters " +
        "WHERE is_processed = 0 AND date BETWEEN " +
        "STR_TO_DATE(?, '%d/%m/%Y') AND " +
        "CURRENT_TIMESTAMP() ORDER BY date ASC;";
}

```

Figure 3.10 Request code for receiving and manipulating email data

AccountDao Implement Class

The AccountDao Implement class implements methods in the AccountDao interface to manage user account data. The queries include

CREATE_ACCOUNT: Creates a new account.

UPDATE_ACCOUNT: Updates account details (e.g., blocking status).

DELETE_ACCOUNT: Updates account data to mark it as deleted.

FIND_ACCOUNT_BY_ID: Finds an account by ID.

FIND_ACCOUNT_BY_NUMBER: Finds an account by its number.

FIND_ALL_ACCOUNTS_BY_USER_ID: Retrieves all accounts for a user.

FIND_ALL_ACCOUNTS: Retrieves all accounts.

SEARCH_BY_CRITERIA: Searches accounts by criteria.

SEARCH_BY_CRITERIA_AND_USER_ID: Searches accounts by criteria for a specific user.

```
public class AccountDaoImpl implements AccountDao {  
    /**  
     * SQL queries  
     */  
    private static final String CREATE_ACCOUNT =  
        "INSERT INTO accounts(user_id, number, balance, currency, is_blocked, is_deleted) " +  
        "VALUES(?, ?, ?, ?, ?, ?)";  
    private static final String UPDATE_ACCOUNT = "UPDATE accounts SET balance = ?, is_blocked = ? WHERE account_id = ?";  
    private static final String DELETE_ACCOUNT = "UPDATE accounts SET is_blocked = 1, is_deleted = 1 WHERE account_id = ?";  
    private static final String FIND_ACCOUNT_BY_ID = "SELECT * FROM accounts WHERE account_id = ? AND is_deleted = 0";  
    private static final String FIND_ACCOUNT_BY_NUMBER = "SELECT * FROM accounts WHERE number = ? AND is_deleted = 0";  
    private static final String FIND_ALL_ACCOUNTS_BY_USER_ID = "SELECT * FROM accounts WHERE user_id = ? AND is_deleted = 0";  
    private static final String FIND_ALL_ACCOUNTS = "SELECT * FROM accounts WHERE is_deleted = 0";  
    private static final String SEARCH_BY_CRITERIA =  
        "SELECT * FROM accounts " +  
        "WHERE is_deleted = 0 AND number LIKE CONCAT(?, '%') AND " +  
        "balance >= ? AND balance <= ? AND " +  
        "currency LIKE CONCAT(?, '%') ORDER BY account_id ASC";  
    private static final String SEARCH_BY_CRITERIA_AND_USER_ID =  
        "SELECT * FROM accounts " +  
        "WHERE user_id = ? AND is_deleted = 0 AND " +  
        "number LIKE CONCAT(?, '%') AND balance >= ? AND balance <= ? AND " +  
        "currency LIKE CONCAT(?, '%') ORDER BY account_id ASC";  
}
```

Figure 3.11 Request code for receiving and manipulating account data

BankCardDao Implement Class

The BankCardDao Implement class implements methods in the BankCardDao interface to manage bank card data. The queries include:

CREATE_CARD: Links a new card to a user's account.

UPDATE_CARD: Updates card details (e.g., activity status).

DELETE_CARD: Unlinks and deletes a card from the system.

FIND_CARD_BY_CARD_ID: Finds a card by its ID.

FIND_CARD_BY_NUMBER: Finds a card by its number.

FIND_CARDS_BY_ACCOUNT_ID: Finds all cards linked to a specific account.

FIND_CARDS_BY_USER_ID: Finds all cards linked to a user.

FIND_ALL_CARDS: Retrieves all cards.

```
public class BankCardDaoImpl implements BankCardDao {  
  
    /**  
     * SQL queries  
     */  
    private static final String CREATE_CARD =  
        "INSERT INTO bank_cards(account_id, number, cvv, validity, is_active) " +  
        "VALUES(?, ?, ?, ?, ?)";  
    private static final String UPDATE_CARD = "UPDATE bank_cards SET is_active = ? WHERE card_id = ?";  
    private static final String DELETE_CARD = "DELETE FROM bank_cards WHERE card_id = ?";  
    private static final String FIND_CARD_BY_CARD_ID = "SELECT * FROM bank_cards WHERE card_id = ?";  
    private static final String FIND_CARD_BY_NUMBER = "SELECT * FROM bank_cards WHERE number = ?";  
    private static final String FIND_CARDS_BY_ACCOUNT_ID = "SELECT * FROM bank_cards WHERE account_id = ?";  
    private static final String FIND_CARDS_BY_USER_ID =  
        "SELECT bank_cards.* FROM bank_cards " +  
        "INNER JOIN accounts ON bank_cards.account_id = accounts.account_id " +  
        "WHERE accounts.user_id = ?";  
    private static final String FIND_ALL_CARDS = "SELECT * FROM bank_cards";  
}
```

Figure 3.12 Request code for receiving and manipulating card data

PaymentDao Implement Class

The PaymentDao Implement class implements methods in the PaymentDao interface to manage payment data. The queries include:

CREATE_PAYMENT: Records a new payment.

FIND_ALL_PAYMENTS: Retrieves all payments in the system.

FIND_PAYMENT_BY_ID: Finds a payment by ID.

FIND_ALL_PAYMENTS_BY_ACCOUNT_ID: Finds all payments for a specific account.

FIND_ALL_PAYMENTS_BY_USER_ID: Finds all payments for a specific user.

FIND_ALL_PAYMENTS_BY_USER_ID_LIMIT_3: Retrieves up to three payments for a user.

SEARCH_BY_CRITERIA: Searches payments by criteria.

SEARCH_BY_CRITERIA_AND_FINAL_DATE_AS_CURRENT_TIMESTAMP: Searches payments by criteria with the end date as the current timestamp.

SEARCH_BY_CRITERIA_WITHOUT_ISOUTGOING: Searches payments by criteria, ignoring payment type.

SEARCH_BY_CRITERIA_WITHOUT_ISOUTGOING_AND_FINAL_DATE_AS_CURRENT_TIMESTAMP: Searches payments by criteria, ignoring payment type and setting the end date as the current timestamp.

```
public class PaymentDaoImpl implements PaymentDao {  
    /**  
     * SQL queries  
     */  
    private static final String CREATE_PAYMENT =  
        "INSERT INTO payments(account_id, isOutgoing, senderNumber, senderAmount, senderCurrency, recipientNumber, recipientAmount, r  
        \"VALUES(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)\";  
    private static final String FIND_ALL_PAYMENTS = "SELECT * FROM payments\";  
    private static final String FIND_PAYMENT_BY_ID = "SELECT * FROM payments WHERE payment_id = ?\";  
    private static final String FIND_ALL_PAYMENTS_BY_ACCOUNT_ID = "SELECT * FROM payments WHERE account_id = ? ORDER BY payment_id DESC\";  
    private static final String FIND_ALL_PAYMENTS_BY_USER_ID =  
        "SELECT payments.* FROM payments " +  
        "INNER JOIN accounts ON payments.account_id = accounts.account_id " +  
        "WHERE accounts.user_id = ? ORDER BY payment_id DESC\";  
    private static final String FIND_ALL_PAYMENTS_BY_USER_ID_LIMIT_3 =  
        "SELECT payments.* FROM payments " +  
        "INNER JOIN accounts ON payments.account_id = accounts.account_id " +  
        "WHERE accounts.user_id = ? ORDER BY payment_id DESC LIMIT 3\";  
    private static final String SEARCH_BY_CRITERIA =  
        "SELECT payments.* FROM payments " +  
        "INNER JOIN accounts ON payments.account_id = accounts.account_id " +  
        "WHERE accounts.user_id = ? AND isOutgoing = ? AND " +  
        "date BETWEEN STR_TO_DATE(?, '%d/%m/%Y %H:%i:%s') AND " +  
        "STR_TO_DATE(?, '%d/%m/%Y %H:%i:%s') ORDER BY date DESC\";  
    private static final String SEARCH_BY_CRITERIA_AND_FINAL_DATE_AS_CURRENT_TIMESTAMP =  
        "SELECT payments.* FROM payments " +  
        "INNER JOIN accounts ON payments.account_id = accounts.account_id " +  
        "WHERE accounts.user_id = ? AND isOutgoing = ? AND date BETWEEN " +  
        "STR_TO_DATE(?, '%d/%m/%Y %H:%i:%s') AND " +  
        "CURRENT_TIMESTAMP() ORDER BY date DESC\";  
    private static final String SEARCH_BY_CRITERIA_WITHOUT_ISOUTGOING =  
        "SELECT payments.* FROM payments " +  
        "INNER JOIN accounts ON payments.account_id = accounts.account_id " +  
        "WHERE accounts.user_id = ? AND date BETWEEN " +  
        "STR_TO_DATE(?, '%d/%m/%Y %H:%i:%s') AND " +  
        "STR_TO_DATE(?, '%d/%m/%Y %H:%i:%s') ORDER BY date DESC\";  
    private static final String SEARCH_BY_CRITERIA_WITHOUT_ISOUTGOING_AND_FINAL_DATE_AS_CURRENT_TIMESTAMP =  
        "SELECT payments.* FROM payments " +  
        "INNER JOIN accounts ON payments.account_id = accounts.account_id " +  
        "WHERE accounts.user_id = ? AND date BETWEEN " +  
        "STR_TO_DATE(?, '%d/%m/%Y %H:%i:%s') AND " +  
        "CURRENT_TIMESTAMP() ORDER BY date DESC\";
```

Figure 3.13 Request code for receiving and manipulating payment data

3.4.4 Development of Database Triggers

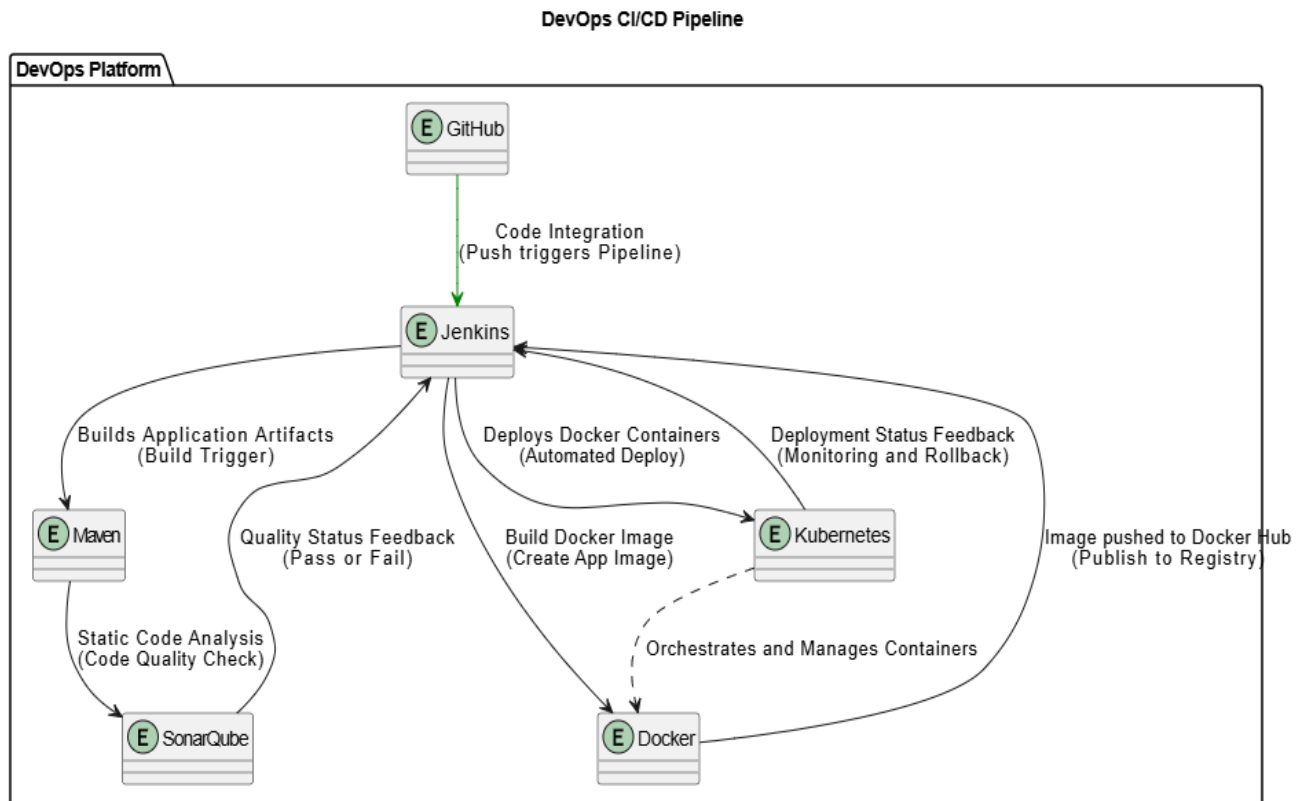
A trigger is a special type of stored procedure that executes automatically in response to specific events in the database. Unlike standard procedures, triggers are not explicitly called by users; instead, they are stored within the database management system (DBMS) and activate automatically based on data modifications within associated tables. Triggers operate independently of the programming language or software used, ensuring a standardized mechanism for handling data changes.

In the development of the payment system database, triggers have been implemented to enhance data integrity and enforce business rules directly at the database level. Key benefits of using triggers in this context include:

- **Data Validation:** Triggers provide a "last line of defense" by validating data before it is committed to the database, ensuring that only accurate and consistent information is stored.
- **Automated Business Logic:** Triggers allow for the automation of business processes that depend on specific data events, minimizing manual intervention and reducing the likelihood of human error.
- **Consistency and Integrity Enforcement:** By enforcing rules at the database level, triggers help maintain the reliability and accuracy of the data, especially in cases where multiple applications or users interact with the same database.

3.5 Deployment Design Using DevOps Tools

Deploying a Payment Management System requires a structured and efficient approach to avoid downtime, security issues, and operational failures. DevOps tools and practices provide a robust framework to streamline deployment, manage environments, and ensure reliability in production. Here's an outline of the DevOps tools suitable for each phase of deployment.



The DevOps pipeline for the Payment Management System integrates multiple tools to streamline code deployment and ensure quality. Code changes are managed in GitHub, where each push triggers the Jenkins CI/CD pipeline, automating the build and deployment process. Jenkins uses Maven to compile and package the code, while SonarQube performs static code analysis, providing feedback on code quality. Once the code passes quality checks, Docker creates a container image, which is stored in Docker Hub for easy access. Finally, Kubernetes orchestrates and manages the deployment, scaling, and monitoring of containerized applications, ensuring the Payment Management System is robust and efficiently deployed.

3.6 USER INTERFACE DESIGN

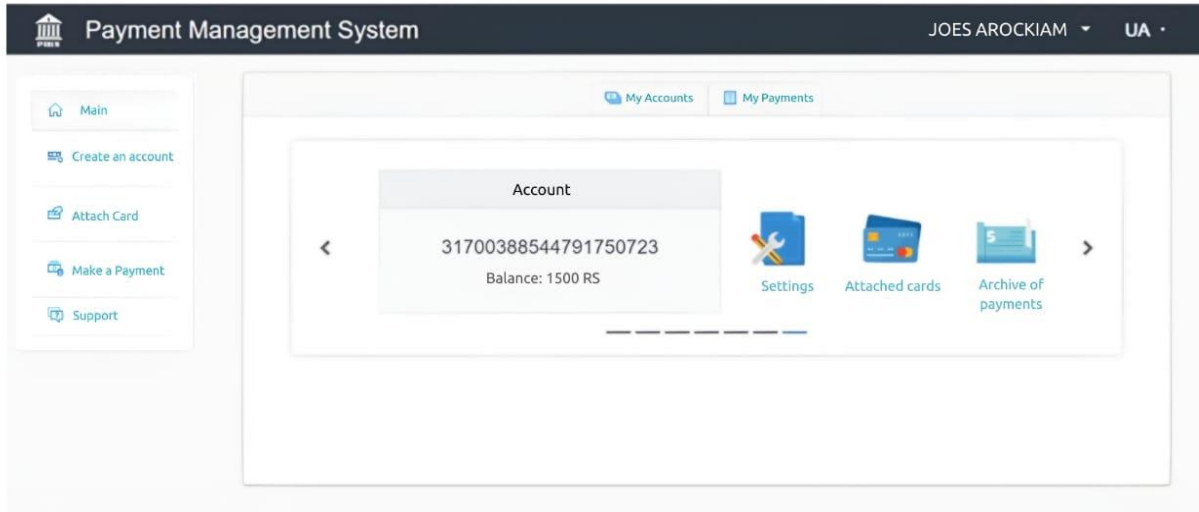
The screenshot shows the login interface of the Payment Management System. At the top, a dark header bar contains the system logo and name on the left, and a language dropdown set to 'EN' on the right. The main content area is light gray and features a central white login card. The card is titled 'LOGIN TO THE PAYMENT MANAGEMENT SYSTEM' and shows a 'STATUS: BETA' in red. Below the title are two buttons: 'BETA-USER' and 'BETA-ADMIN'. The login form includes a phone number field with a dropdown for the country code (set to '+91') and the number '9345883907', which is marked as 'Correct' with a green checkmark. Below this is a password field with masked characters '*****', also marked as 'Correct'. A blue 'SIGN IN' button is positioned below the password field. At the bottom of the card, there are two links: 'Forgot your password? Recovery' and 'Don't have an account yet? Sign Up'.

Screen 3.1 User and Admin login

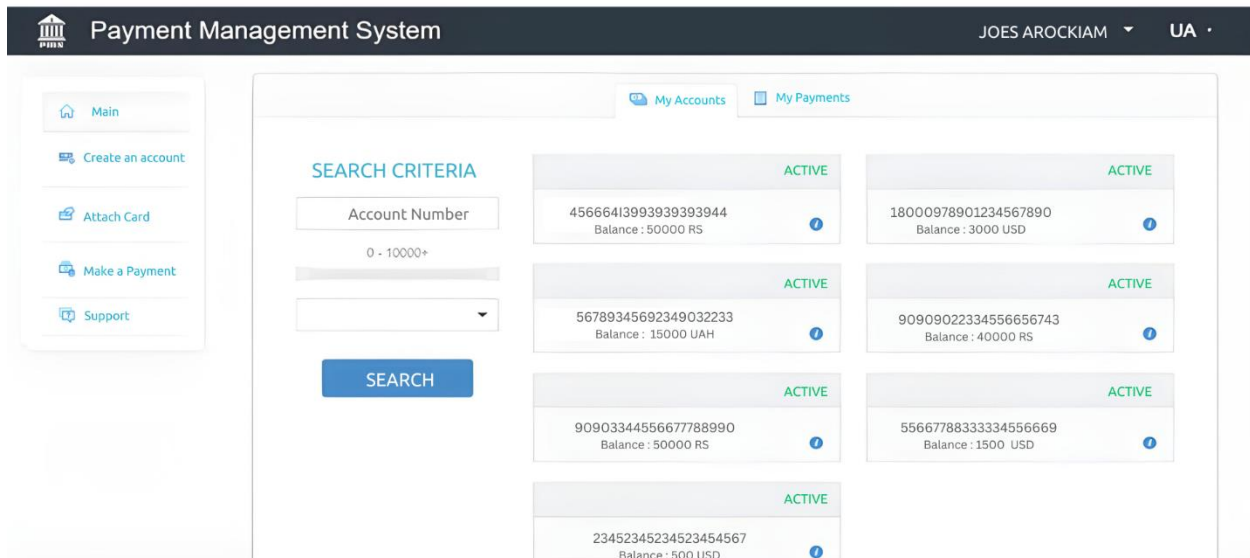
Screen 3.1 shows the desktop view of user login

The screenshot shows the registration interface of the Payment Management System. The layout is consistent with the login page, featuring a dark header bar with the system logo and name, and a language dropdown set to 'EN'. The main content area is light gray and features a central white registration card. The card is titled 'REGISTRATION'. The form includes several fields: a first name field with 'Joes Arockiam' (marked 'Correct'), a last name field with 'X' (marked 'Correct'), a phone number field with a dropdown for the country code (set to '+91') and the number '9345883907' (marked 'Correct'), and an email field with 'joesarockiam@gmail.com' (marked 'Correct'). Below these are two password fields, both with masked characters '*****' and marked 'Correct'. A blue 'SIGN UP' button is positioned below the password fields. At the bottom of the card, there is a link: 'Have account already? Login'.

Screen 3.2 User Registration page



Screen 3.3 Main page of the site for an authorized client



Screen 3.4 All accounts of the user

Screen 3.4 shows a page where all user accounts are available for viewing. In addition, it has the ability to search by criteria.

Payment Management System

JOES AROCKIAM UA

Main

Create an account

Attach Card

Make a Payment

Support

ATTACH NEW CARD

Select an account

96475051487965073340
correct

4654 6549 8959 5920
correct

526
correct

Validity period 04 2020

ATTACH CARD

Screen 3.5 Attach a New card to the Account

Screen 3.5 shows the page for attaching a card to an account, which shows the case when the user specifies a card that has already expired.

Payment Management System

JOES AROCKIAM UA

Main

Create an account

Attach Card

Make a Payment

Support

CREATE A NEW PAYMENT

43654448499494942222

Account Number 3345677891123456 Account Holder

Amount 1200

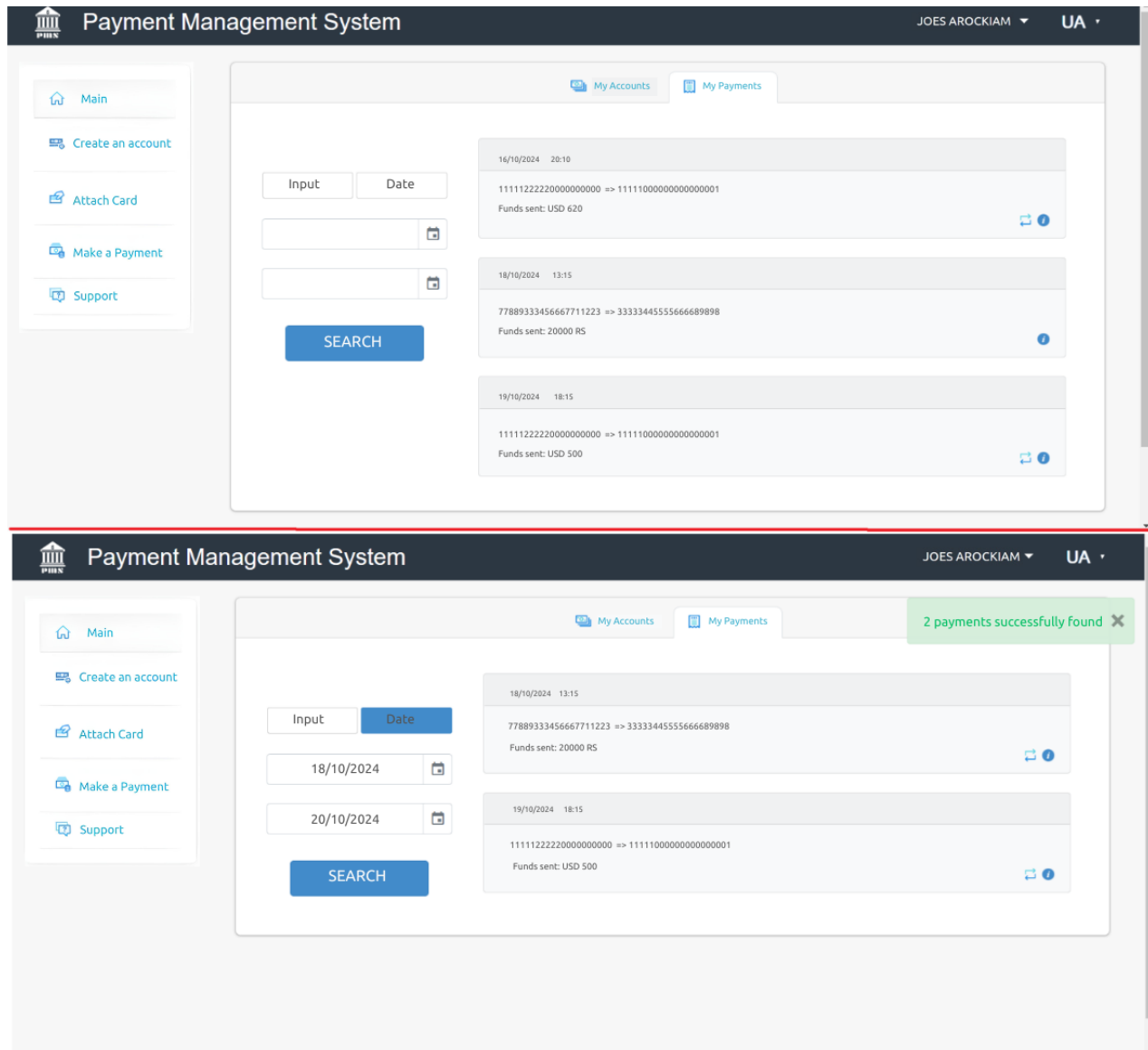
Purpose of payment

MAKE A PAYMENT

FAILED! Not enough money in account.

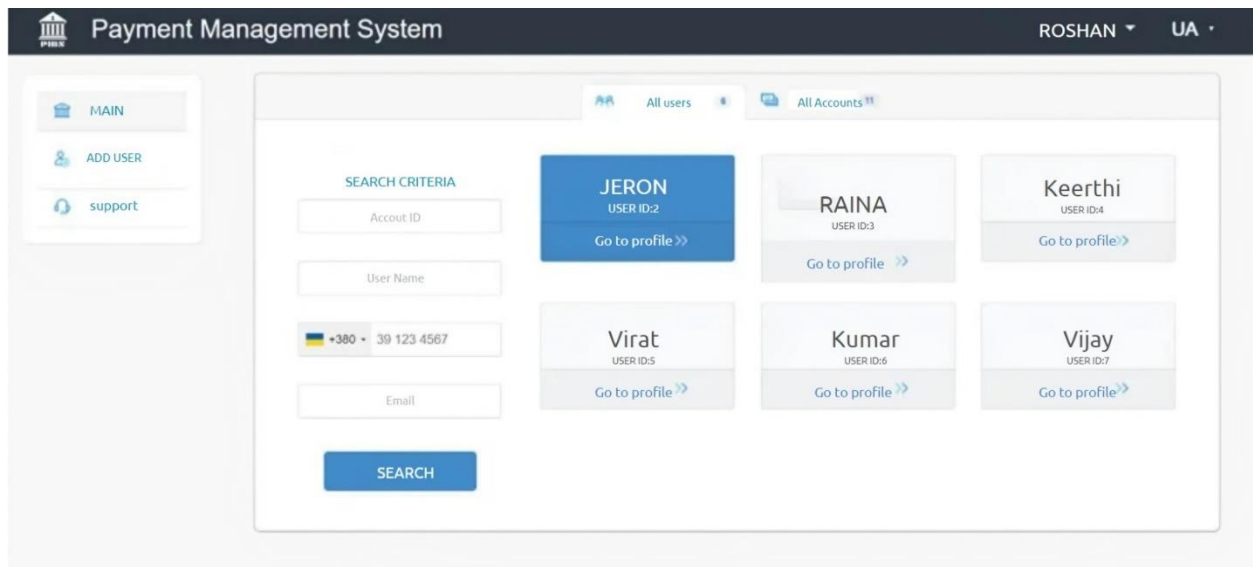
Screen 3.6 Payment page

Screen 3.6 shows the payment page, which shows a case where the selected user account does not have enough funds to send them to another account



Screen 3.7 Page for viewing all user payments

Screen 3.7 shows a page where all user payments are available for viewing. In addition, it has the ability to search by criteria.



Screen 3.8 Main page of the site for an authorized administrator

Screen 3.8 shows the main page of the site, for an authorized user who is an administrator. In addition, it has the ability to search by criteria.

CHAPTER IV

SYSTEM TESTING

System testing is a generic metric of performing a variety of tests on a system to explore functionality or to identify problems. A series of systematic procedures are referred to test how the system should perform and where common mistakes may be found by entering data that may cause the system to malfunction or return incorrect information. The purpose of testing is quality assurance, verification and validation, or reliability estimation. A Test Plan documents the strategy that will be used to verify and ensure that a product meets its design specifications.

4.1 TEST CASE AND TEST REPORTS

Test cases are built around the requirements and specifications i.e., what the system is supposed to do. Test cases are generally driven from the external descriptions of the framework and design parameters.

Each test case contains item criteria such as

PASS

All expected results are achieved and/or all unexpected events are resolved.

FAIL

Testing process response does not confirm the expected results.

Table 1 – User Management Tests

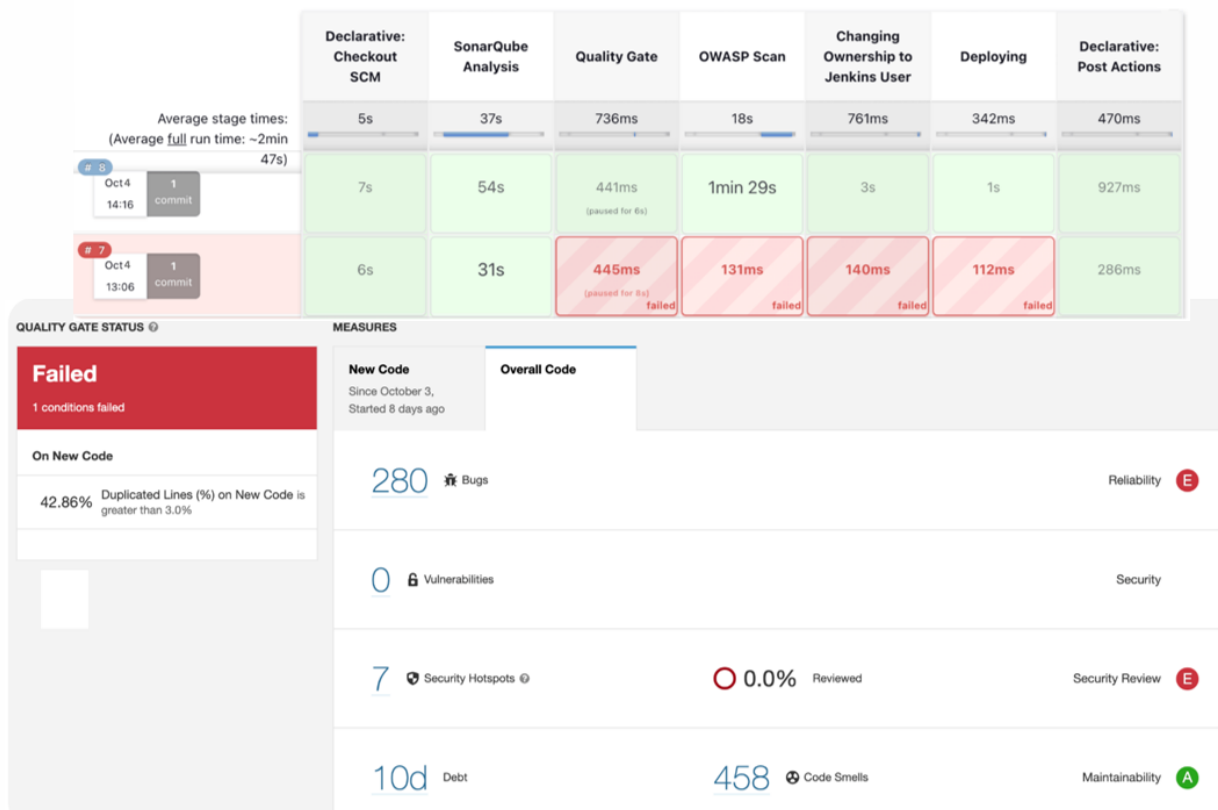
Test Case ID	Test Scenario	Test Steps	Expected Outcome	Result
T1	User Login	Enter valid login credentials	User navigates to dashboard	PASS
T2	User Login	Enter invalid login credentials	Login fails, error message displayed	FAIL
T3	User Registration	Register with valid details	Registration successful, user can log in	PASS
T4	User Registration	Register with invalid details	Registration fails, error message displayed	FAIL
T5	Password Reset	Enter valid email for reset	Password reset link sent to email	PASS
T6	Password Reset	Enter invalid email for reset	Password reset fails, error message displayed	FAIL
T7	Update Profile	Enter updated profile info	Profile updated successfully	PASS
T8	View Profile	Access profile page	Profile details displayed	PASS
T9	Account Deletion	Enter valid user ID for deletion	Account deleted successfully	PASS
T10	Account Deletion	Enter invalid user ID for deletion	Account deletion fails, error message shown	FAIL

Table -2 Transaction Management Tests

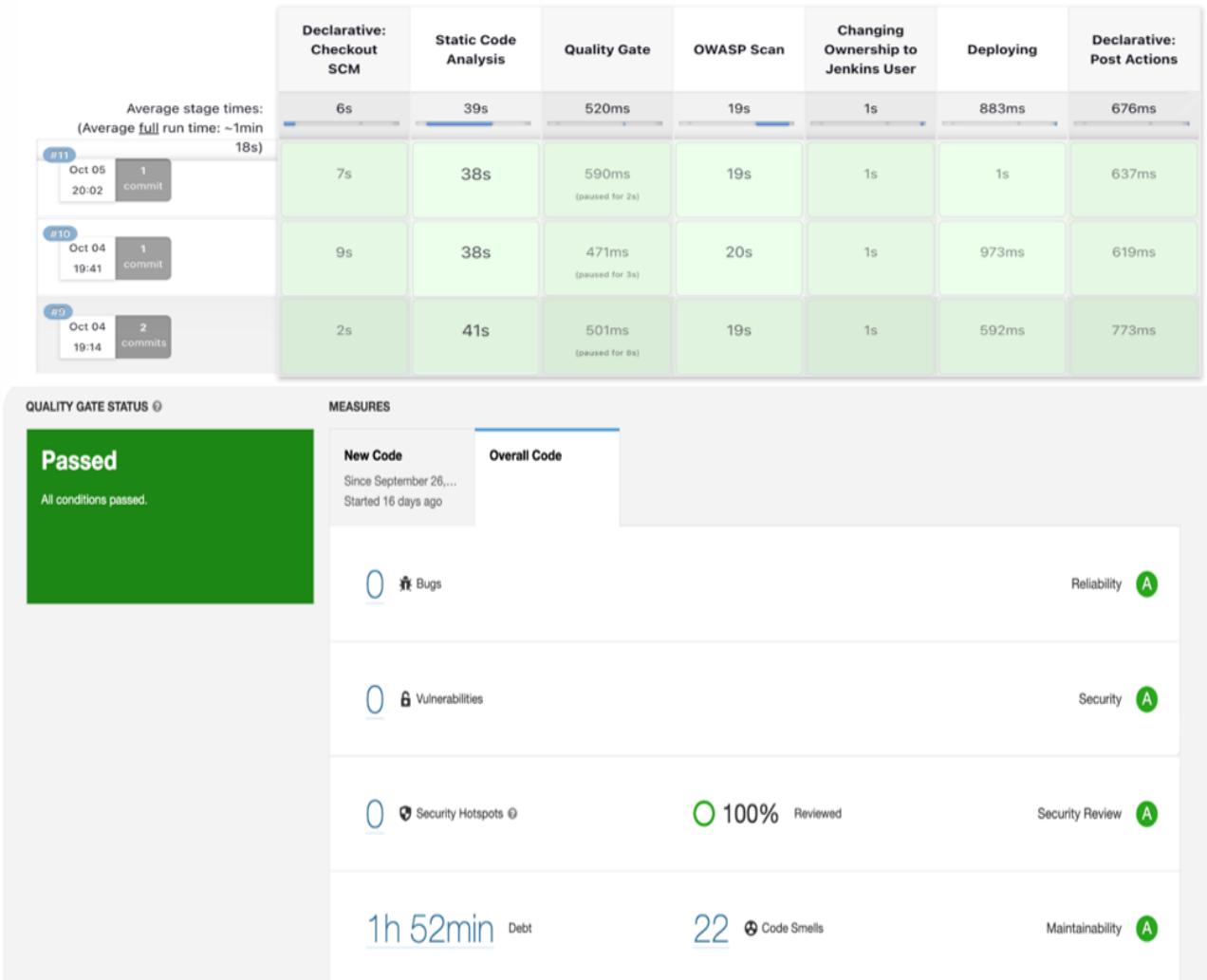
Test Case ID	Test Scenario	Test Steps	Expected Outcome	Result
T11	Payment Processing	Enter valid payment details	Payment processed successfully	PASS
T12	Payment Processing	Enter invalid payment details	Payment fails, error message displayed	FAIL
T13	Refund Processing	Enter valid transaction ID	Refund processed successfully	PASS
T14	Refund Processing	Enter invalid transaction ID	Refund fails, error message displayed	FAIL
T15	Transaction History Viewing	Access transaction history page	Transaction history displayed	PASS
T16	Transaction History Viewing	Access history with invalid ID	History not displayed, error shown	FAIL
T17	Admin Payment Approval	Approve pending payment	Payment approved successfully	PASS
T18	Admin Payment Approval	Approve with invalid payment ID	Approval fails, error message displayed	FAIL
T19	Generate Payment Invoice	Generate invoice with valid ID	Invoice generated successfully	PASS
T20	Generate Payment Invoice	Generate invoice with invalid ID	Invoice generation fails, error shown	FAIL

4.2 JENKINS AND SONARQUBE TESTING

Jenkins and SonarQube testing are essential for maintaining high standards of code quality, security, and reliability in software development. Jenkins automates the build, test, and deployment processes, ensuring that new code changes are integrated smoothly and without delay. SonarQube complements this by analyzing the code for potential issues like bugs, vulnerabilities, code duplication, and security hotspots. Together, these tools allow teams to detect and address issues early in the development lifecycle, reducing technical debt and enhancing code maintainability. Integrating Jenkins and SonarQube in a CI/CD pipeline helps ensure that only high-quality, secure code moves forward, leading to a more robust and resilient software product.



4.2.1 Failed Jenkins and SonarQube testing image



4.2.2 Passed Jenkins and SonarQube testing image

CHAPTER V

SYSTEM IMPLEMENTATION

Implementation is the process of converting a newly designed or revised system into an operational one. This process includes all steps necessary to guide end-users on how to use the new system, deploying the system into production, ensuring that all essential data is accurate and available, and validating that business functions interacting with the system work as expected.

In this project, the Payment Management System is implemented using DevOps tools and services like AWS EC2, MySQL, Jenkins, Docker, and Git. The system is deployed on an EC2 instance, with data stored in a MySQL database, managed through Java-based backend services. The DevOps pipeline automates the build, test, and deployment processes, ensuring continuous integration and delivery.

INSTALLATION PROCEDURE

- Open the terminal and navigate to the project directory.
- Build the application using Maven and Jenkins:
 - Run `mvn clean install` to compile and package the application.
 - Use Jenkins to automate the build and deployment process.
- Start the Docker container for the MySQL database:
Run `docker-compose up -d` to initialize the MySQL instance and required services.
- Deploy the application to the AWS EC2 instance:
Run `java -jar target/payment-management-system.jar` to start the application

REQUIREMENTS ON SERVER SIDE

- Operating System: Ubuntu 20.04 LTS, 64-bit
- Hosting Platform: Amazon Web Services (AWS)
- Database: MySQL (Hosted on AWS RDS or local instance)
- Java Version: Java 8 (for compatibility with existing libraries)
- Build & Deployment Tools: Jenkins, Docker.

CHAPTER VI

CONCLUSION

In conclusion, the Global Payment Management System is a comprehensive solution designed to streamline payment processing and management tasks, ensuring accuracy, efficiency, and security in financial transactions. Currently in the testing phase, the system leverages a robust architecture with tools like AWS, Docker, Jenkins, and MySQL to enable a scalable and reliable environment for handling payment data. This project aims to provide a secure, user-friendly platform that can be accessed by clients to manage their payments efficiently. The system's modular design allows for future adaptability and expansion, making it well-suited to meet the evolving needs of its users.

6.1 FEATURE OF DEVELOPED SYSTEM

As the system advances toward live deployment, several key improvements and enhancements are planned:

- **Full Deployment on AWS:** Upon successful completion of the testing phase, the system will be fully deployed on AWS servers to ensure global accessibility and high availability, providing users with a stable and responsive platform.
- **Enhanced Security Measures:** Implement advanced security protocols, such as multi-factor authentication and data encryption, to further protect sensitive payment information and comply with industry standards.
- **Scalability and Performance Optimization:** Optimize the system's performance to handle increased transaction volumes, enabling it to scale seamlessly as user demand grows.
- **Integration with Third-party Services:** Explore integration options with other financial services, payment gateways, and CRM systems to provide users with a comprehensive financial management experience.
- **Mobile Accessibility:** Develop a mobile-friendly interface or dedicated mobile app to allow users to access and manage their payments on the go, enhancing user accessibility.

BIBLIOGRAPHY

WEB REFERENCE

- [1] <https://www.jenkins.io/doc/> - Jenkins Documentation, Continuous Integration and Continuous Delivery concepts, October 5, 2023.
- [2] <https://docs.docker.com/> - Docker Documentation, Installation, and Containerization basics, November 2, 2023.
- [3] <https://aws.amazon.com/getting-started/> - AWS Getting Started, Cloud Services, and Deployment on AWS, October 28, 2023.
- [4] <https://docs.oracle.com/en/java/> - Java Documentation, Programming Concepts, and Language Reference, September 18, 2023.
- [5] <https://dev.mysql.com/doc/> - MySQL Documentation, Database Management and SQL queries, October 10, 2023.
- [6] https://www.tutorialspoint.com/design_pattern/data_access_object_pattern.htm - Data Access Object (DAO) Design Pattern, Overview and Implementation, October 22, 2023.